

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Título del Proyecto: Sistema de Facturación y Pagos ITGSA

PONDERACIÓN: 30 pts

 **Tiempo estimado: 50 hrs**

Índice

1. Marco Formativo	3
1.1. Valor	3
1.2. Competencia(s)	3
1.3. Habilidad(es) blandas a formar	3
2. Resultado del Aprendizaje	4
2.1. Objetivo SMART	4
3. Resumen Ejecutivo	4
4. Enunciado del Proyecto	4
4.1 Descripción del problema o necesidad a resolver	4
Entrada: config.xml	5
Entrada: transac.xml	6
Programa 1 — Frontend (ASP.NET Razor Pages)	6
Servicio 2 — Backend (ASP.NET WebAPI)	7
4.2 Alcance del proyecto	8
4.3 Recursos y herramientas a utilizar	9
4.4 Entregables	9
5. Material de apoyo	10
6. Metodología	10
7. Cronograma	11
8. Rúbrica de Calificación	12
8.1 Requisitos para optar a la calificación	12
8.2 Detalle de la Calificación	13
8.3 Comentarios Generales	15

1. Marco Formativo

1.1. Valor

Nombre del Valor	¿Cómo se aplica en el proyecto?
Responsabilidad	El estudiante debe diseñar, implementar y entregar un sistema funcional respetando restricciones tecnológicas, plazos y estándares académicos definidos.
Excelencia académica	Se espera que la solución aplique correctamente POO, arquitectura en capas y buenas prácticas de desarrollo de software.

1.2. Competencia(s)

Con la elaboración de este proyecto el estudiante adquirirá la siguiente competencia:

Diseñar e implementar un sistema de información distribuido compuesto por un servicio backend (API REST en ASP.NET WebAPI) y una aplicación frontend (ASP.NET Razor Pages), integrando bases de datos en XML, procesamiento de archivos, lógica de negocio orientada a objetos y comunicación mediante el protocolo HTTP.

1.3. Habilidad(es) blandas a formar

El proyecto le permitirá desarrollar las siguientes habilidades:

- Gestión del tiempo: planificación por releases semanales.
- Comunicación técnica: redacción de documentación en formato de ensayo.
- Resolución de problemas: diseño e implementación de la lógica de abono automático a facturas.
- Autodirección: el estudiante define de forma autónoma los endpoints y estructuras de datos adicionales necesarios.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Implementar un sistema de facturación y pagos con API REST (.NET) y frontend Razor Pages que procese archivos XML y gestione saldos de clientes.	El sistema debe procesar correctamente el 100 % de los casos definidos en los archivos config.xml y transac.xml, superando todos los criterios de la rúbrica.	Utilizando ASP.NET WebAPI para el backend y ASP.NET Razor Pages para el frontend, aplicando POO y versionamiento en GitHub con mínimo 4 releases.	Para desarrollar la competencia de construcción de sistemas distribuidos con comunicación HTTP, persistencia en XML y lógica de negocio compleja.	Antes del jueves 30 de abril a las 11:59 pm, con entregas parciales semanales documentadas como releases en GitHub.

3. Resumen Ejecutivo

La empresa Industria Típica Guatemalteca, S.A. (ITGSA) requiere un backend para gestionar las facturas generadas por su Tienda Virtual y los pagos efectuados a través de las bancas en línea de los bancos con que opera. El sistema desarrollado en este proyecto consta de dos componentes: un Servicio 2 (backend) implementado como una API REST con ASP.NET WebAPI en .NET, que expone endpoints para grabar configuraciones, procesar transacciones, consultar estados de cuenta y resumir ingresos; y un Programa 1 (frontend) construido con ASP.NET Razor Pages, que actúa como cliente del backend y permite al operador cargar archivos XML de configuración y transacciones, consultar estados de cuenta por cliente y visualizar gráficas de ingresos por banco. La persistencia de datos se realiza mediante archivos XML. La solución está desarrollada bajo el paradigma de programación orientada a objetos y utiliza versionamiento en GitHub.

4. Enunciado del Proyecto

4.1 Descripción del problema o necesidad a resolver

ITGSA opera un canal de distribución en línea (Tienda Virtual) y ha firmado convenios con bancos guatemaltecos para recibir pagos mediante Banca Virtual. El problema central es que no existe un sistema que centralice y controle el flujo de facturas y pagos: cuando un cliente realiza un pago, este debe aplicarse automáticamente a su factura más antigua pendiente; si el pago excede el saldo adeudado, el remanente queda como saldo a favor y se descuenta en la siguiente compra. Se requiere además un frontend para que el personal de ITGSA cargue archivos de configuración (clientes y bancos) y de transacciones (facturas y pagos), y consulte el estado de cuenta de cada cliente y los ingresos consolidados por banco.

El frontend a construir debe ser capaz de recibir dos tipos de archivo de entrada: uno para configurar los bancos y clientes del backend, y otro para gestionar transacciones de facturación o pagos generadas por la Tienda Virtual o la Banca Virtual respectivamente.

Entrada: config.xml

Este archivo contiene el listado de clientes y bancos que trabajan con ITGSA. Es incremental: se pueden ingresar varios archivos de configuración para ampliar la base de datos de clientes y bancos. Si se repite un NIT de cliente o un código de banco, los datos deben ser actualizados.

```
<?xml version="1.0"?>
<config>
  <clientes>
    <cliente>
      <NIT> [valor_alfanumérico] </NIT>
      <nombre> [valor_alfanumérico] </nombre>
    </cliente>
    ...
  </clientes>
  <bancos>
    <banco>
      <codigo> [valor_numérico] </codigo>
      <nombre> [valor_alfanumérico] </nombre>
    </banco>
    ...
  </bancos>
</config>
```

Como respuesta a cada archivo config.xml cargado al sistema, se deberá generar un XML con la siguiente estructura:

```
<?xml version="1.0"?>
<respuesta>
  <clientes>
    <creados> [valor_numérico] </creados>
    <actualizados> [valor_numérico] </actualizados>
  </clientes>
  <bancos>
    <creados> [valor_numérico] </creados>
    <actualizados> [valor_numérico] </actualizados>
  </bancos>
</respuesta>
```

Entrada: transac.xml

Este archivo contiene un listado de facturas generadas por la Tienda Virtual y un listado de pagos generados por las Bancas virtuales. Es incremental: se pueden ingresar varios archivos de transacciones para crear nuevas facturas y nuevos pagos. No puede haber actualizaciones de datos de facturas o de pagos.

```
<?xml version="1.0"?>
<transacciones>
  <facturas>
    <factura>
      <numeroFactura> [valor_alfanumérico] </numeroFactura>
      <NITcliente> [valor_alfanumérico] </NITcliente>
      <fecha> [dd/mm/yyyy] </fecha>
      <valor> [valor_numérico] </valor>
    </factura>
    ...
  </facturas>
  <pagos>
    <pago>
      <codigoBanco> [valor_numérico] </codigoBanco>
      <fecha> [dd/mm/yyyy] </fecha>
      <NITcliente> [valor_alfanumérico] </NITcliente>
      <valor> [valor_numérico] </valor>
    </pago>
    ...
  </pagos>
</transacciones>
```

Como respuesta a cada archivo transac.xml cargado al sistema, se deberá generar un XML con la siguiente estructura:

```
<?xml version="1.0"?>
<transacciones>
  <facturas>
    <nuevasFacturas> [valor_numérico] </nuevasFacturas>
    <facturasDuplicadas> [valor_numérico] </facturasDuplicadas>
    <facturasConError> [valor_numérico] </facturasConError>
  </facturas>
  <pagos>
    <nuevosPagos> [valor_numérico] </nuevosPagos>
    <pagosDuplicados> [valor_numérico] </pagosDuplicados>
    <pagosConError> [valor_numérico] </pagosConError>
  </pagos>
</transacciones>
```

Programa 1 — Frontend (ASP.NET Razor Pages)

Este programa consiste en una aplicación web que simula la aplicación principal. Contiene únicamente las funcionalidades necesarias para testear el buen funcionamiento del backend (Servicio 2). En esta aplicación se podrán mostrar los eventos que se procesarán y los datos estadísticos almacenados en la base de datos XML.

Para realizar el frontend deberá utilizarse ASP.NET Razor Pages.

Componentes del Programa 1:

- **Resetear datos:** esta opción mandará la instrucción al API para devolver el sistema al estado inicial, es decir, sin datos.
- **Cargar archivo de configuración:** se desplegará una pantalla para gestionar la carga de los archivos config.xml. Luego de cargar el archivo deberá presentar la respuesta apropiada.
- **Cargar archivo de transacciones:** se desplegará una pantalla para gestionar la carga de los archivos transac.xml. Luego de cargar el archivo deberá presentar la respuesta apropiada.
- **Peticiones:** en este apartado se deben tener las siguientes opciones:
- **Consultar estado de cuenta:** al seleccionar esta opción se debe solicitar un NIT de cliente, o si desea ver a todos los clientes ordenados por NIT, y presentar la siguiente información de forma amigable:

```
Cliente: 399399-K - Almacén Guatemalteco
Saldo actual: Q. 0.00
Transacciones (ordenadas de la más reciente a la más antigua)
Fecha          Cargo          Abono
31/03/2024          Q 300.00 (BI)
28/03/2024      Q. 300.00 (Fact. #10)
```

- **Consultar ingresos:** al seleccionar esta opción se debe solicitar un mes y mostrar una gráfica de “Ingresos por banco” con los últimos 3 meses en forma descendente (por ejemplo: marzo/2024, febrero/2024, enero/2024).
- **Ayuda:** desplegará dos opciones: una para visualizar información del estudiante y otra para visualizar la documentación del programa.

NOTA 1:

Todos los reportes y gráficas deben poder ser generados en formato .pdf

NOTA 2:

El Frontend deberá mostrar los datos recuperados del consumo del servicio que se describe a continuación.

Servicio 2 — Backend (ASP.NET WebAPI)

Este servicio consiste en una API REST que brindará servicios utilizando el protocolo HTTP. Su funcionalidad principal es procesar los datos recibidos del Programa 1 y almacenarlos en uno o varios archivos XML. También tiene la funcionalidad de devolver los datos procesados para que sean mostrados según se indica en la sección de Archivos de Entrada y de Respuesta.

Para la realización de este servicio debe utilizarse .NET. El estudiante deberá definir por su propia cuenta los métodos y endpoints que necesitará. Los endpoints sugeridos son:

- POST /grabarConfiguracion — recibe y procesa config.xml (clientes y bancos).
- POST /grabarTransaccion — recibe y procesa transac.xml (facturas y pagos).
- POST /limpiarDatos — resetea el sistema al estado inicial.

- GET /devolverEstadoCuenta — retorna el estado de cuenta de un cliente o todos los clientes.
- GET /devolverResumenPagos — retorna el resumen de ingresos por banco de los últimos 3 meses.

Lógica de negocio clave:

- Los pagos se deben abonar a la factura más antigua del cliente que aún tenga saldo pendiente.
- Si el pago excede el total adeudado, el cliente queda con un saldo a favor, el cual se aplica automáticamente en la siguiente compra.

NOTA:

Durante la calificación, el Servicio 2 podrá ser consumido directamente desde Postman y deberá retornar una respuesta XML con los datos necesarios para cada funcionalidad.

4.2 Alcance del proyecto

Alcance obligatorio:

- Backend (Servicio 2): API REST en ASP.NET WebAPI (.NET) con endpoints para grabar configuración, grabar transacciones, resetear datos, consultar estado de cuenta y consultar resumen de ingresos.
- Frontend (Programa 1): aplicación web en ASP.NET Razor Pages que consume el backend mediante HTTP.
- Procesamiento de archivos XML de entrada: config.xml y transac.xml.
- Generación de respuestas XML con la estructura definida en el enunciado.
- Lógica de abono automático a la factura más antigua y manejo de saldo a favor.
- Persistencia de datos en archivos XML.
- Versionamiento en GitHub con mínimo 4 releases.
- Documentación en formato de ensayo (mínimo 4, máximo 7 páginas) con diagrama de clases y diagrama entidad-relación.

Alcance opcional:

- Exportación de reportes y gráficas a PDF desde el frontend.
- Diagramas adicionales (actividades, conceptual) en los apéndices de la documentación.

4.3 Recursos y herramientas a utilizar

Tipo	Categoría	Descripción
Obligatorio	Plataforma/Framework	ASP.NET WebAPI (.NET) — Backend REST
Obligatorio	Plataforma/Framework	ASP.NET Razor Pages (.NET) — Frontend web
Obligatorio	IDE	Visual Studio / Visual Studio Code (IDEs aprobados por el laboratorio).
Obligatorio	Plataforma	GitHub — Control de versiones y releases
Obligatorio	Formato de datos	XML — Archivos de entrada (config.xml, transac.xml) y persistencia de datos
Opcional	Software	Herramienta de generación de PDF (iTextSharp, DinkToPdf, PuppeteerSharp)-
Opcional	Software	Postman — Pruebas independientes del backend

4.4 Entregables

Tipo	Descripción
Código fuente	Repositorio GitHub con nombre IPC2_Proyecto3_#Carnet, que contenga el backend (Servicio 2) y el frontend (Programa 1) con mínimo 4 releases.
Backend	API REST funcional en ASP.NET WebAPI con todos los endpoints requeridos.
Frontend	Aplicación web funcional en ASP.NET Razor Pages que consume el backend.
Archivos XML	Archivos de respuesta generados con la estructura definida en el enunciado (respuesta de config.xml y transac.xml).
Documentación	Ensayo en formato del curso (4–7 páginas) que incluya diagrama de clases y diagrama entidad-relación del modelo XML. Subido al repositorio en carpeta separada.
Entrega UEDI	Archivo de texto con el link del repositorio subido en UEDI.

5. Material de apoyo

Categoría	Link	Descripción
Documentación Oficial	https://learn.microsoft.com/en-us/aspnet/core/web-api	Guía oficial de ASP.NET WebAPI.
Documentación Oficial	https://learn.microsoft.com/en-us/aspnet/core/razor-pages	Guía oficial de ASP.NET Razor Pages.
Documentación Oficial	https://learn.microsoft.com/en-us/dotnet/standard/linq/xml-overview	Manejo de XML con LINQ to XML en .NET.
Documentación Oficial	https://docs.github.com/en/repositories/releasing-projects-on-github	Creación de releases en GitHub.
Manual	https://www.postman.com/downloads/	Herramienta para probar endpoints del backend de forma independiente.

6. Metodología

A continuación se presenta una metodología lógica para abordar el proyecto:

Pasos a seguir:

Paso 1: Análisis y diseño

Leer el enunciado completo e identificar todas las entidades del sistema (Cliente, Banco, Factura, Pago). Diseñar el diagrama de clases y el diagrama entidad-relación del modelo XML antes de escribir código.

Paso 2: Configuración del repositorio y estructura del proyecto

Crear el repositorio en GitHub con el nombre IPC2_Proyecto3_#Carnet. Agregar al auxiliar como colaborador. Crear dos proyectos .NET en la misma solución: uno de tipo ASP.NET WebAPI (Servicio 2) y otro de tipo ASP.NET Razor Pages (Programa 1).

Paso 3: Implementar el modelo de datos XML y las clases de dominio (Release 1)

Definir la estructura de los archivos XML de persistencia. Implementar las clases de dominio (POO) que representen Cliente, Banco, Factura y Pago. Implementar los métodos de lectura y escritura de XML.

Paso 4: Implementar los endpoints del backend (Release 2)

Desarrollar los endpoints requeridos: POST /grabarConfiguracion, POST /grabarTransaccion, POST /limpiarDatos, GET /devolverEstadoCuenta y GET /devolverResumenPagos. Validar con Postman que las respuestas XML cumplen la estructura del enunciado. Implementar la lógica de abono a la factura más antigua y el manejo de saldo a favor.

Paso 5: Implementar el frontend (Release 3)

Desarrollar las páginas Razor: carga de config.xml, carga de transac.xml con visualización de respuesta, consulta de estado de cuenta (por NIT o todos), gráfica de ingresos por banco ('ultimos 3 meses) y sección de ayuda. Conectar cada página al backend mediante HttpClient.

Paso 6: Exportación a PDF, pruebas finales y documentación (Release 4)

Implementar la exportación de reportes y gráficas a PDF. Ejecutar pruebas integrales con distintos archivos XML de entrada. Redactar la documentación en formato de ensayo e incluirla en el repositorio en carpeta separada. Realizar el release final antes de la fecha de entrega.

Recomendaciones:

- Utilizar expresiones regulares para extraer el NIT y las fechas de los archivos XML, descartando información irrelevante según lo indicado en las consideraciones.
- Mantener la lógica de negocio en la capa de servicios del backend, separada de los controladores.
- Realizar commits frecuentes y etiquetarlos correctamente para facilitar la creación de los 4 releases.
- Probar el backend de forma independiente con Postman antes de integrarlo con el frontend.

7. Cronograma

Tipo / Release	Fecha Inicio	Fecha Fin
Release 1 — Modelo de datos y clases de dominio.	Semana 1	Semana 1
Release 2 — Endpoints del backend funcionales.	Semana 2	Semana 2
Release 3 — Frontend integrado con el backend.	Semana 3	Semana 3
Release 4 (Final) — Pruebas, PDF y documentación	Semana 4	Semana 4

8. Rúbrica de Calificación

En esta sección se aborda todo lo relacionado a la calificación, lo que permite claridad para el estudiante y el tutor académico de los aspectos a evaluar y los mismos que conocen desde el inicio por lo que todo proyecto debe incluirlos desde el inicio.

8.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, debe cumplir con los requisitos que se indiquen en esta sección, de no cumplir usted tendrá una nota con valor de cero puntos.

Tema	Descripción	Cumple (Sí/No)
Entrega en UEDI	Subir archivo de texto con el link del repositorio antes del 30 de abril a las 11:59 pm.	
Documentación	Entregar ensayo en formato del curso (4–7 páginas) con diagrama de clases y diagrama E-R. Subido en el repositorio en carpeta separada.	
Uso de POO	El sistema debe implementar programación orientada a objetos. De lo contrario, no habrá derecho a calificación.	
Repositorio GitHub	Repositorio con nombre IPC2_Proyecto3_#Carnet con mínimo 4 releases y auxiliar agregado como colaborador antes de la calificación.	
Tecnología obligatoria	Backend implementado en ASP.NET WebAPI y frontend en ASP.NET Razor Pages.	
Derecho a examen final	Obtener 10/10 en carga de archivos, 5/5 en endpoint grabar configuración y 5/5 en endpoint grabar transacciones.	

8.2 Detalle de la Calificación

	Criterio/Nivel			Nivel de evaluación		
No.	Criterio de evaluación	Punteo maximo		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1	Habilidades					
1.1	Carga De Archivos De Entrada	10	Descripción	Ambos archivos (config.xml y transac.xml) son leídos, parseados y procesados correctamente.	Uno o ambos archivos no se leen o el parsing es incorrecto.	
			Rango del punteo	10 - 7/	6 - 0/	
1.2	Lógica de abono a factura más antigua y manejo de saldo a favor	11	Descripción	Los pagos se aplican correctamente a la factura más antigua; el saldo a favor se gestiona y aplica en compras posteriores.	La lógica de abono o el saldo a favor no funcionan correctamente.	
			Rango del punteo	11 - 7/	6 - 0/	
1.3	Frontend — Consulta de estado de cuenta y gráfica de ingresos	10	Descripción	Se muestra el estado de cuenta por NIT / todos los clientes y la gráfica de los últimos 3 meses correctamente.	La consulta o la gráfica presentan errores o no están implementadas.	
			Rango del punteo	10 - 7/	6 - 0/	
1.4	Frontend — Exportación a PDF y navegación intuitiva	9	Descripción	Los reportes y gráficas se exportan a PDF correctamente; la interfaz es clara e incluye sección de ayuda.	La exportación no funciona o la interfaz es confusa.	
			Rango del punteo	9 - 6/	5 - 0/	
	Sub-Total de Puntos	40				

2	Conocimientos					
2.1	Backend — Endpoints grabar configuración y transacciones (POST)	10	Descripción	Los endpoints procesan correctamente los archivos XML y retornan la respuesta XML con la estructura requerida.	Los endpoints no funcionan o la respuesta XML no cumple la estructura.	
			Rango del punteo	10 - 7/	6 - 0/	
2.2	Backend — Endpoints estado de cuenta y resumen de ingresos (GET)	10	Descripción	Los endpoints retornan XML con estado de cuenta ordenado cronológicamente y resumen de ingresos por banco.	Los datos retornados son incorrectos o faltan campos requeridos.	
			Rango del punteo	10 - 7/	6 - 0/	
2.3	Backend — Endpoint resetear datos y manejo de duplicados/errores	10	Descripción	El sistema reinicia correctamente y detecta facturas/pagos duplicados o con error, reportándolos en el XML de respuesta.	El reseteo no funciona o los duplicados/errores no se detectan.	
			Rango del punteo	10 - 7/	6 - 0/	
2.4	Uso correcto de POO — diseño de clases coherente con la solución	15	Descripción	Las clases reflejan correctamente las entidades del dominio, con encapsulamiento, herencia y/o polimorfismo donde aplique.	El código no aplica POO o el diseño de clases es inconsistente con la solución.	
			Rango del punteo	15 - 10/	9 - 0/	
2.5	Documentación — formato, diagrama de clases y diagrama E-R	15	Descripción	El ensayo cumple el formato del curso, incluye diagrama de clases completo y diagrama E-R del modelo XML.	La documentación está incompleta, tiene formato incorrecto o falta algún diagrama obligatorio.	
			Rango del punteo	15 - 10/	9 - 0/	

	Sub-Total de Puntos	60				
	Total	100				

8.3 Comentarios Generales
